Lab 05

Build, Test, and Package Applications with IntelliJ IDEA and Docker

Name: Saba Gull

Registration No: 2023-BSCS-074

Course Code: CS-649 (Cloud DevOps)

Section: A

GitHub Username: sabagu003-ui

Repository Links

Submission repository: https://github.com/sabagu003-ui/DevOps

Lab 05 directory: https://github.com/sabagu003-ui/DevOps/tree/main/Labs/Lab05

IntelliJ Docker repository: https://github.com/WaqasSaleem97/Intellij-Idea-docker

Task 1: Clone and Launch IntelliJ IDEA

Step 1: Cloned the IntelliJ Docker repository on the Ubuntu host.

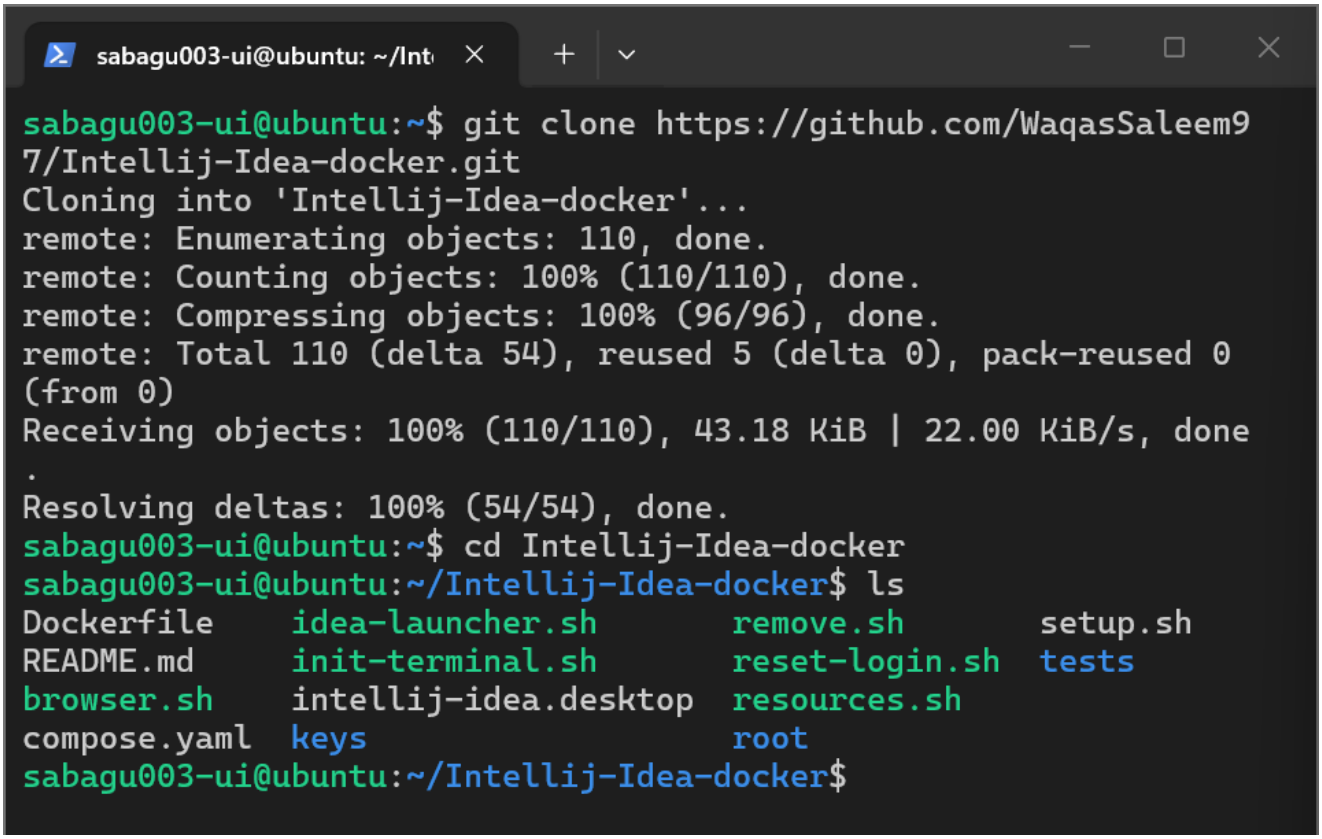


Figure 1.1: Cloned the Intellij-Idea-docker repository (intellij_repo_clone.png)

Step 2: Started the container with setup.sh; docker compose ps shows the intellij service running on port 3001.

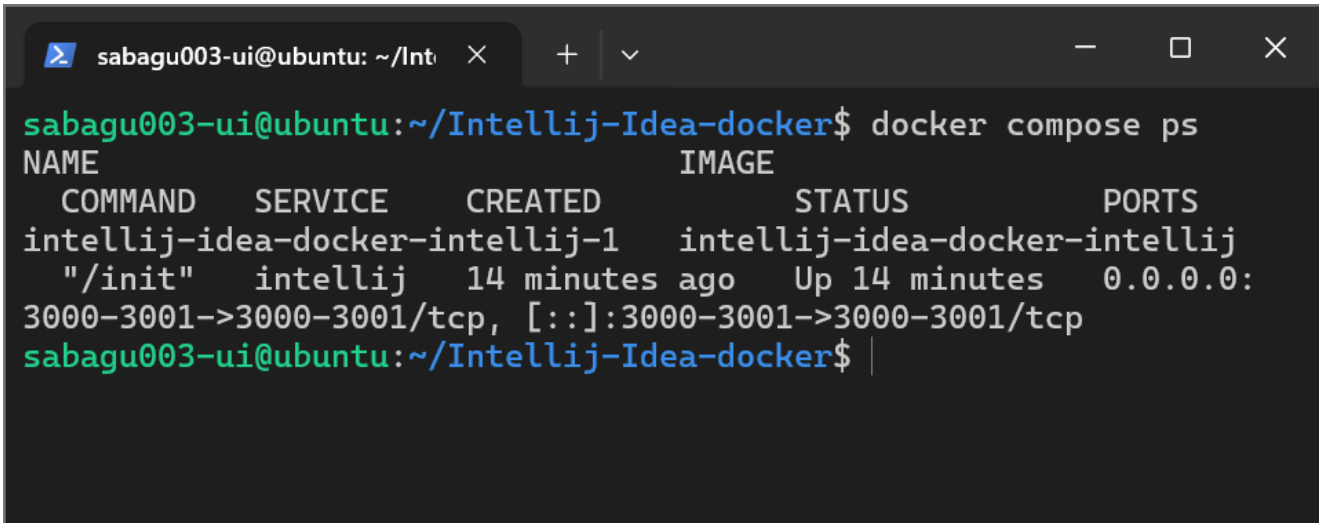


Figure 1.2: Intellij service running (docker compose ps) (intellij_running.png)

Step 3: Opened https://localhost:3001 and reached the IntelliJ IDEA Welcome screen in the browser desktop.

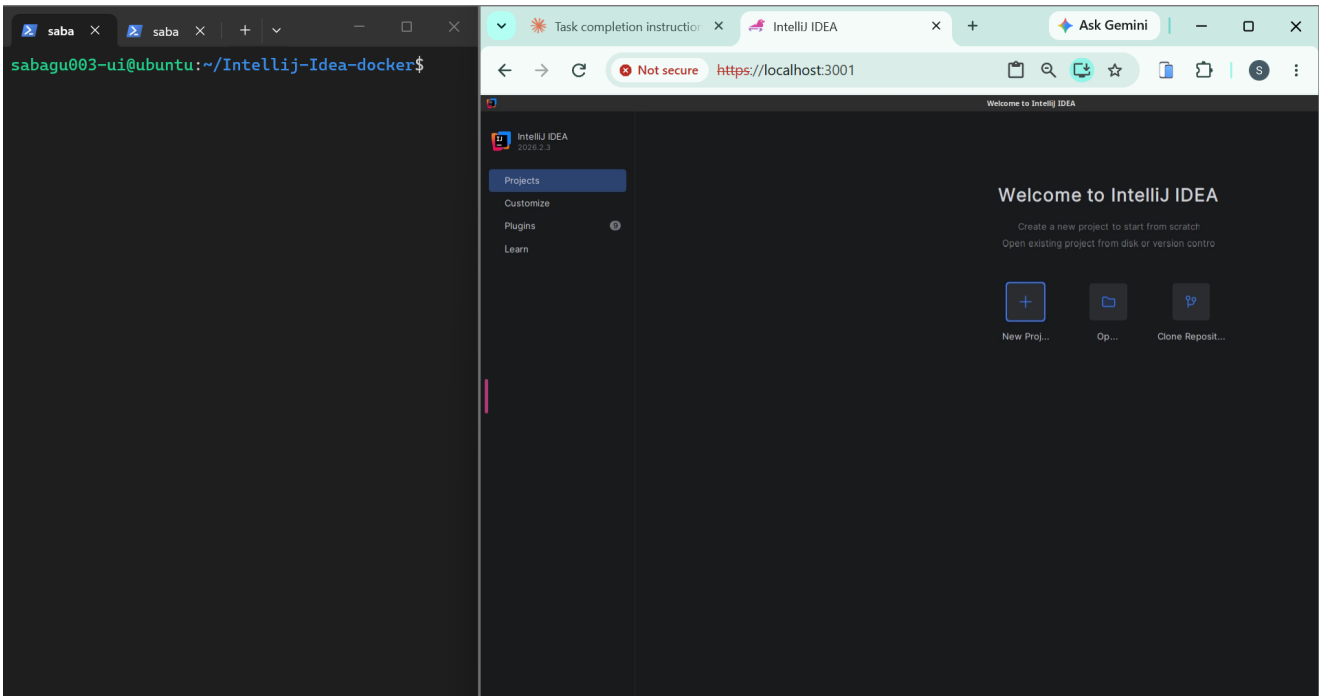


Figure 1.3: IntelliJ IDEA running in the browser desktop (intellij_desktop.png)

Task 2: Build and Run the Maven Project

Step 1: Cloned java-maven-app into /config/IdeaProjects using IntelliJ's Clone Repository.

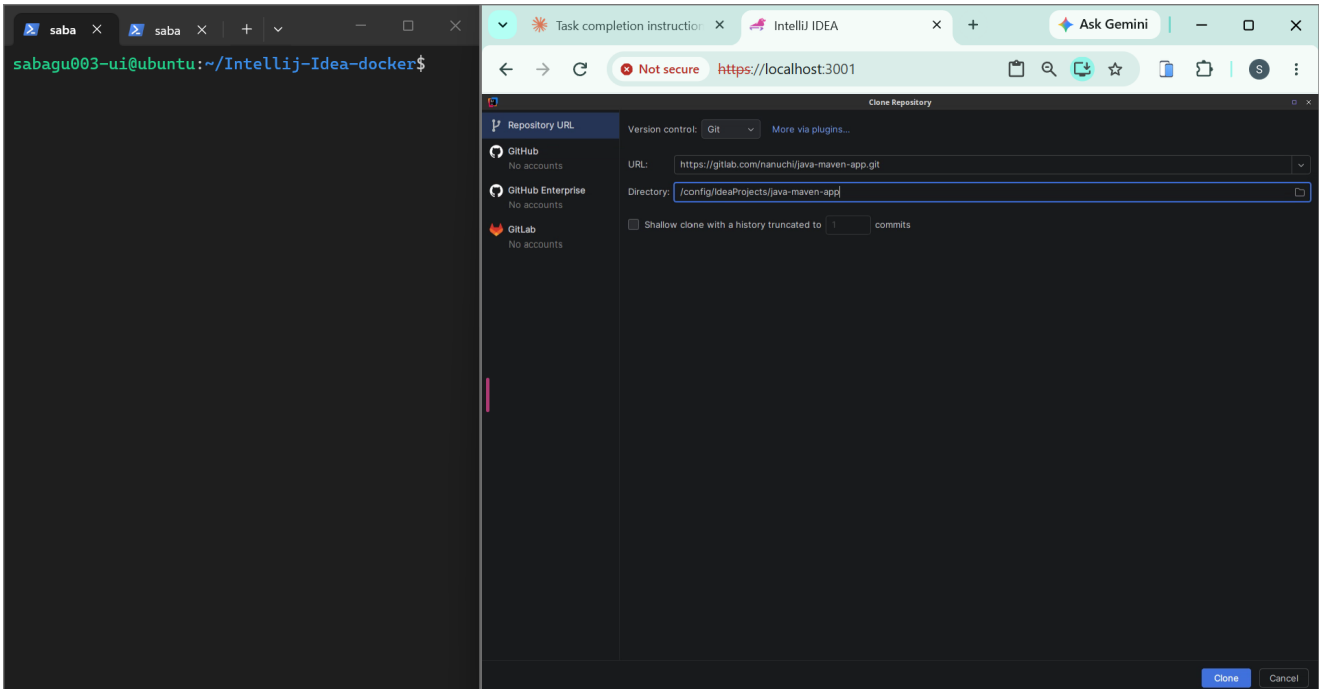


Figure 2.1: Clone dialog for java-maven-app (maven_project_clone.png)

Step 2: Installed Maven from the IntelliJ terminal and checked its version.

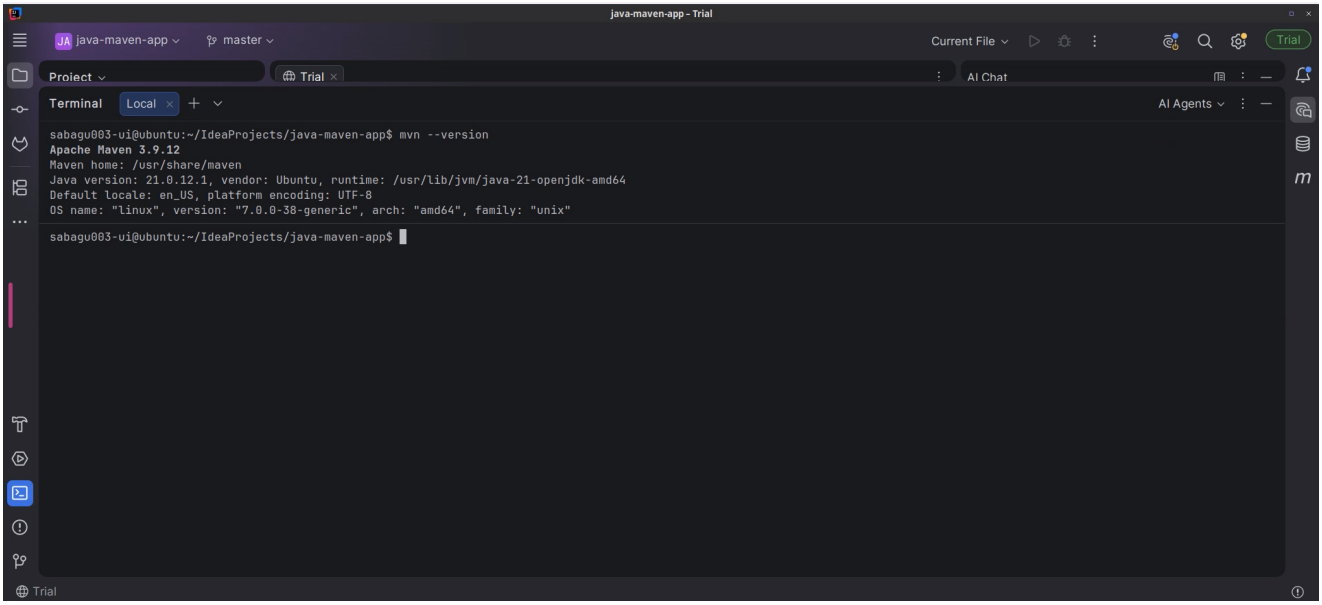


Figure 2.2: Maven installed (mvn --version) (maven_installed.png)

Step 3: Packaged the application with mvn package.

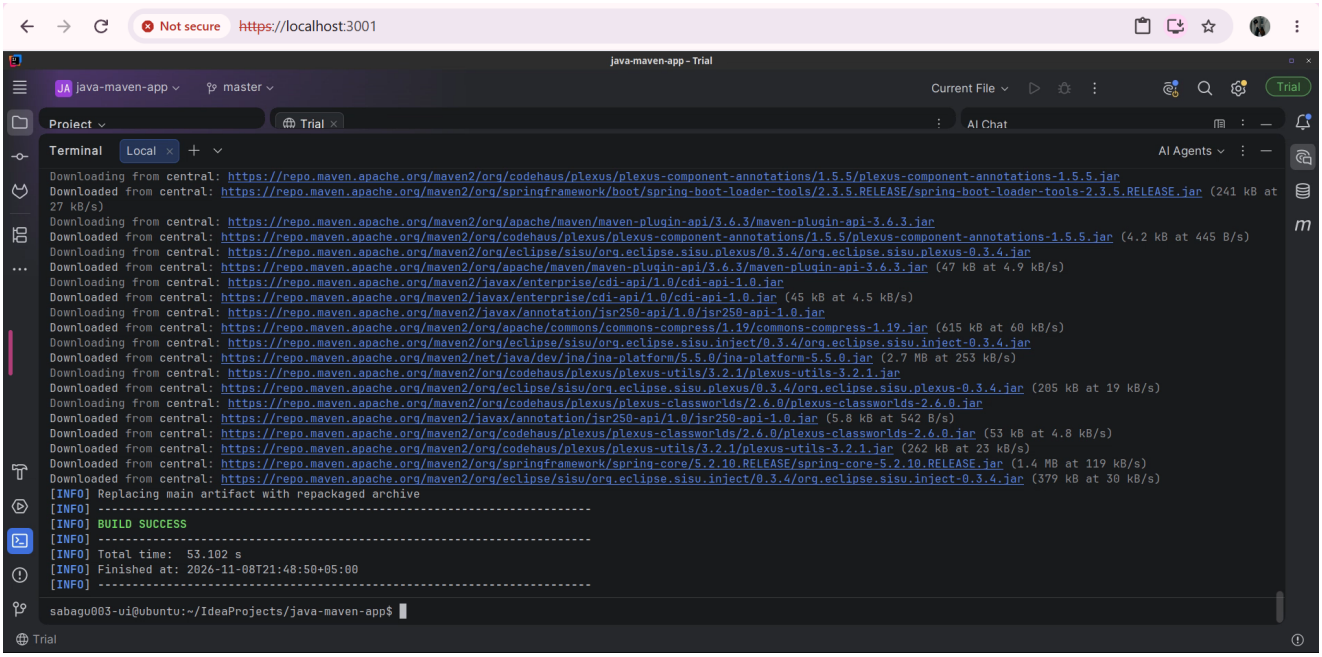


Figure 2.3: mvn package - BUILD SUCCESS (maven_package.png)

Step 4: Added the Elasticsearch 7.0.0 dependency to pom.xml and synced Maven.

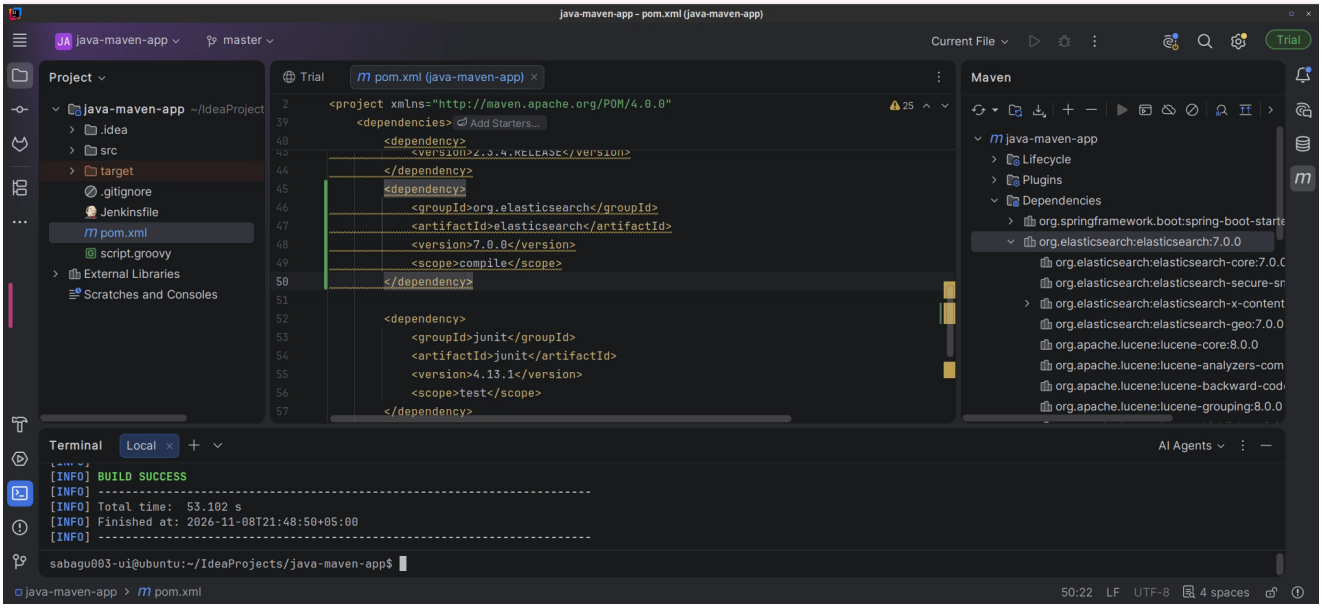


Figure 2.4: Elasticsearch 7.0.0 dependency in pom.xml (maven_elasticsearch_dependency.png)

Step 5: Packaged again after adding the dependency.

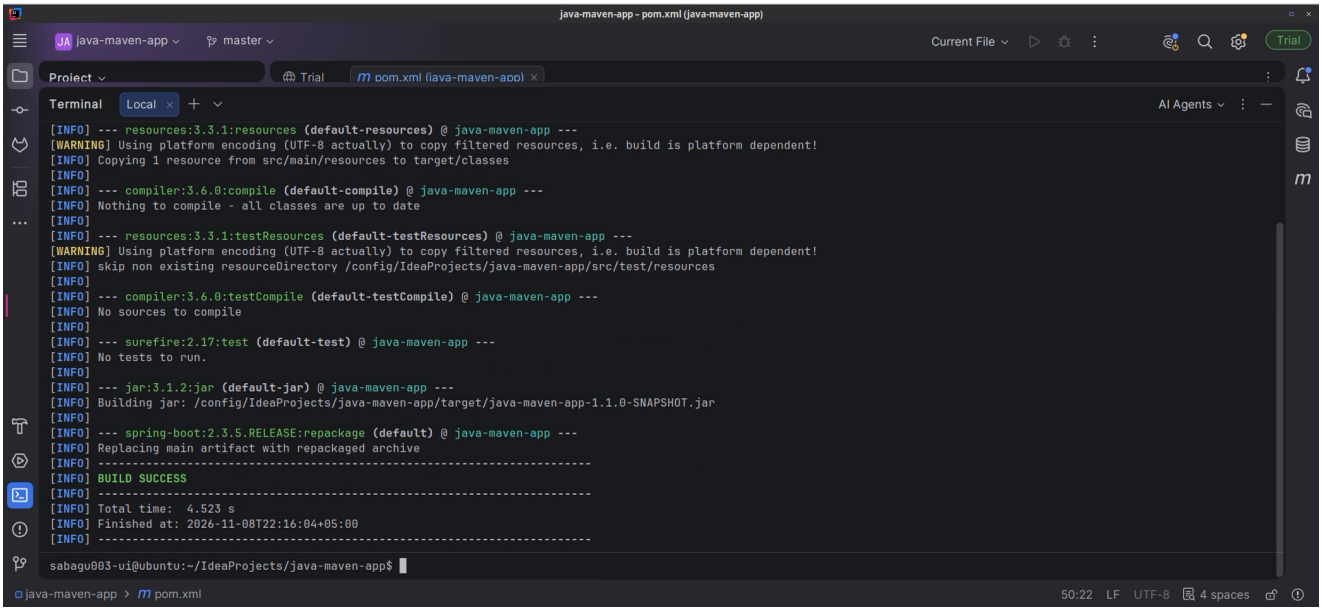


Figure 2.5: mvn package after the dependency - BUILD SUCCESS (maven_repackage.png)

Step 6: Ran mvn test. The sample has no tests, so Maven reports No tests to run.

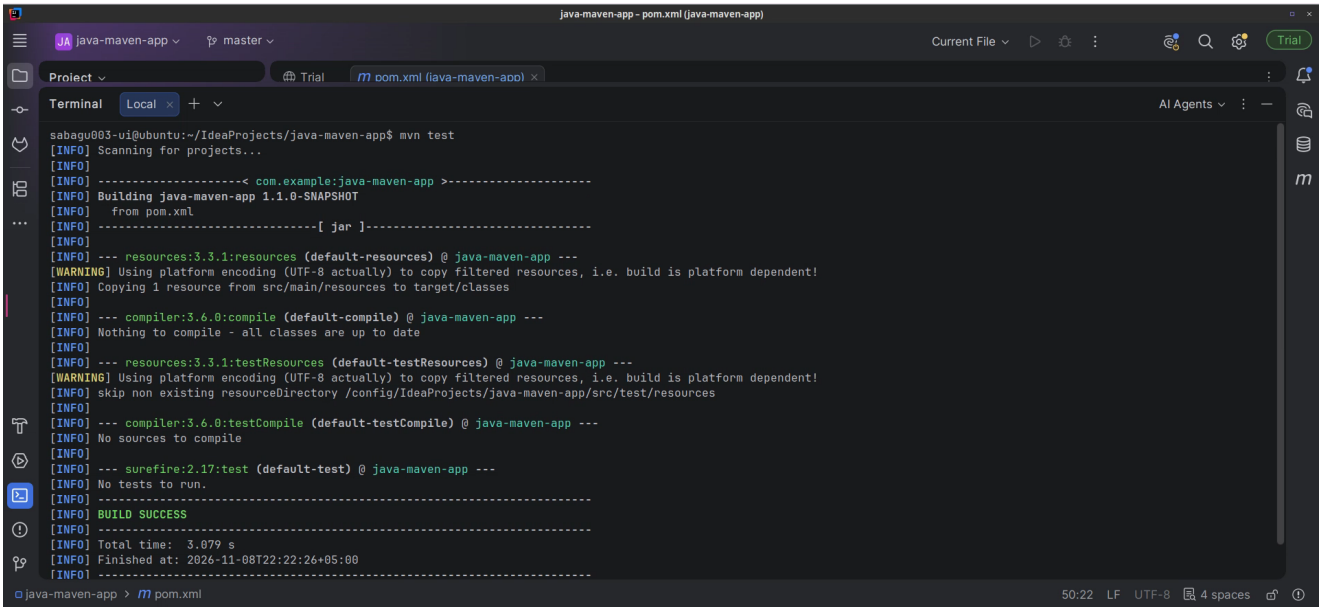


Figure 2.6: mvn test result (maven_test.png)

Step 7: Located the generated JAR in the target directory.

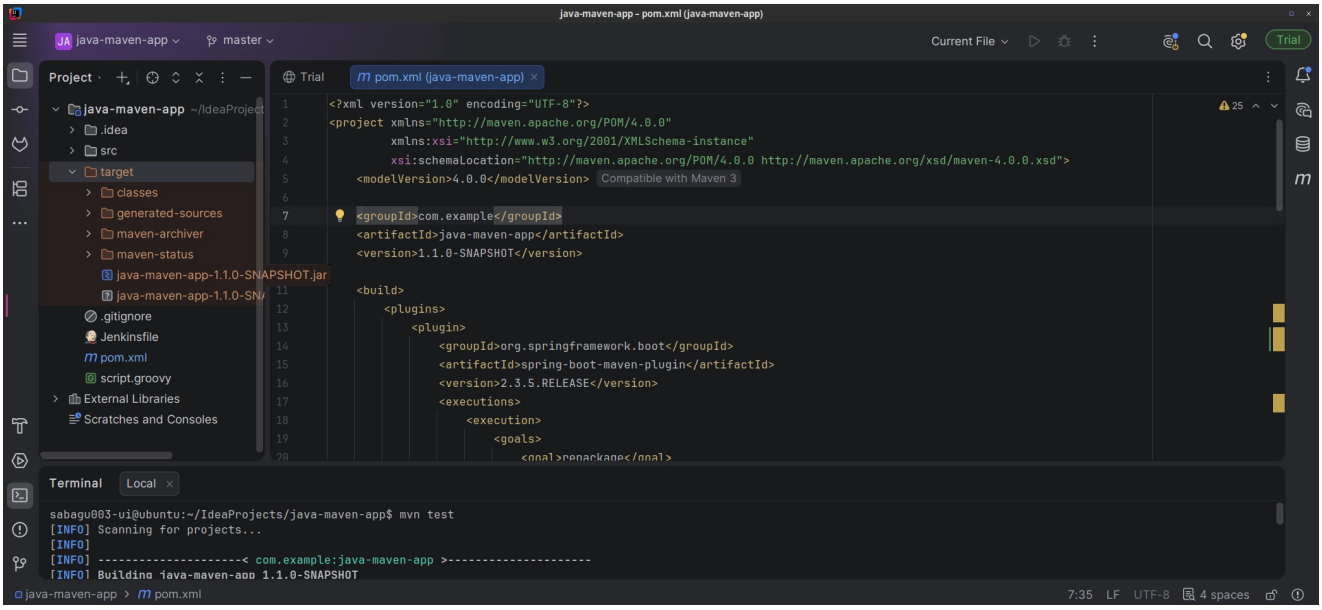


Figure 2.7: java-maven-app-1.1.0-SNAPSHOT.jar in target (maven_jar.png)

Step 8: Ran the JAR with java -jar; Spring Boot started on port 8080.

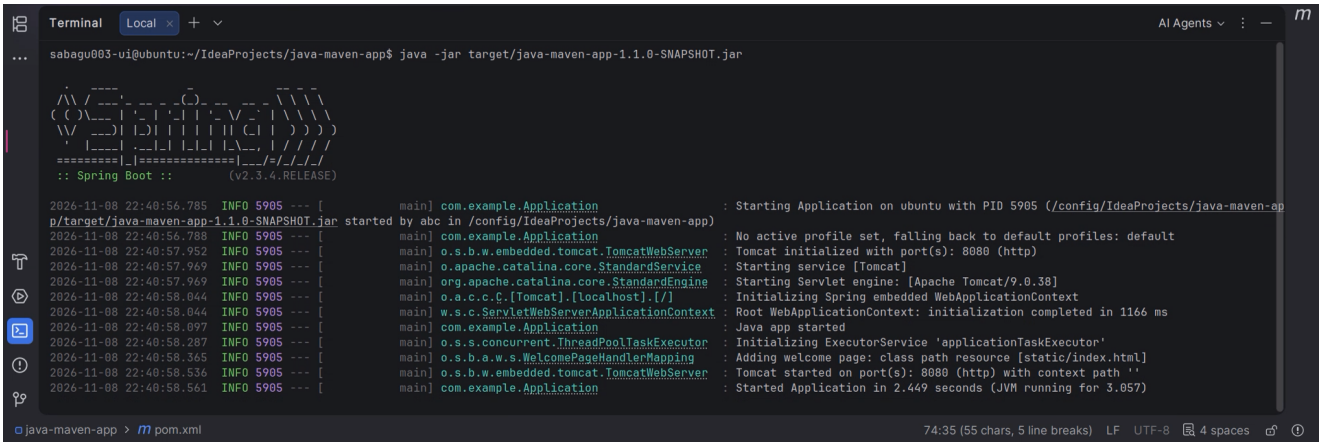


Figure 2.8: Maven application running (maven_running.png)

Step 9: Stopped the application with Ctrl+C and the prompt returned.

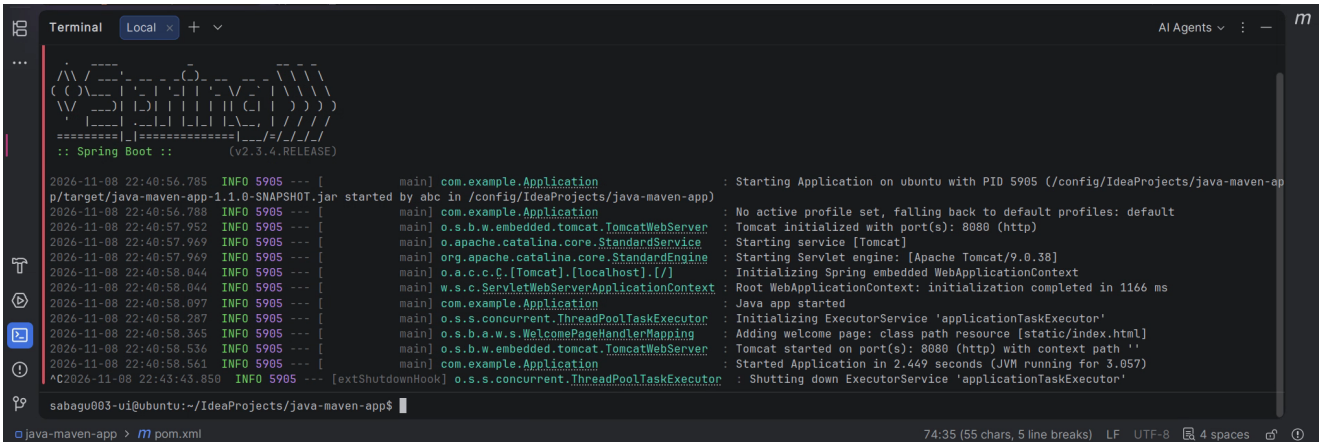


Figure 2.9: Maven application stopped (maven_stopped.png)

Step 10: Closed the project and returned to the Welcome screen.

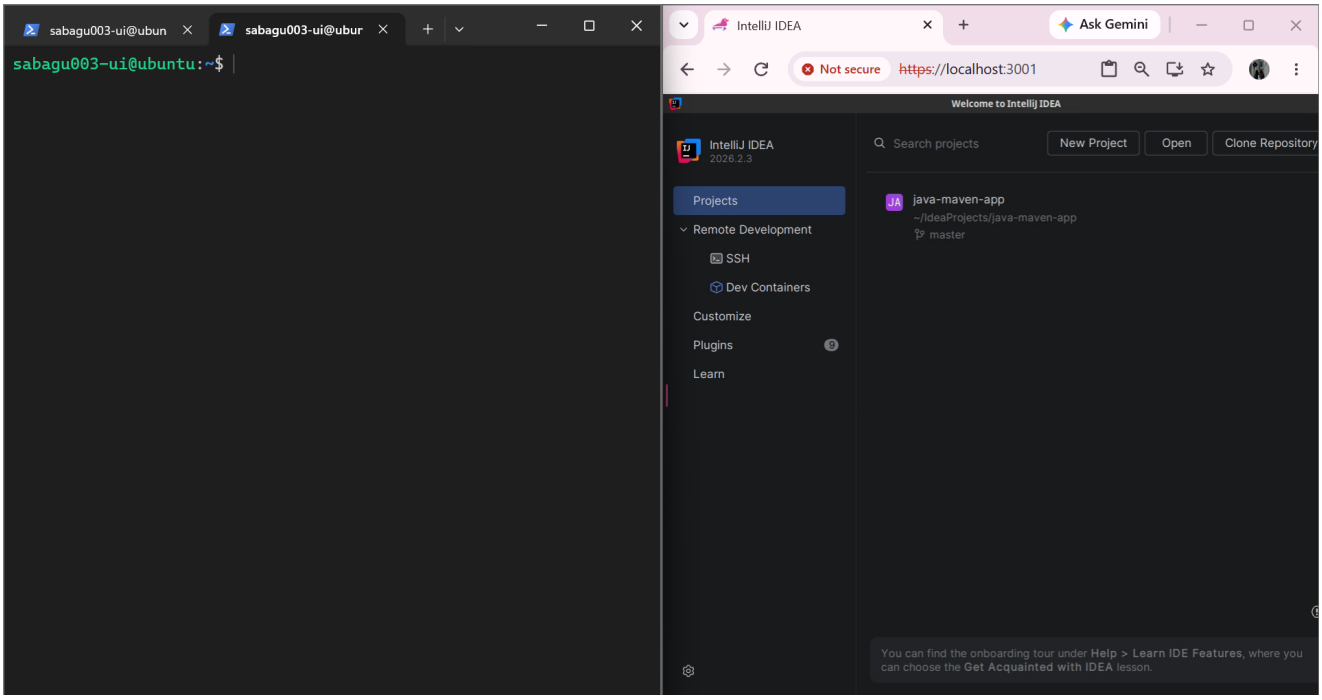


Figure 2.10: Maven project closed (maven_project_closed.png)

Task 3: Build and Run the Gradle Project

Step 1: Cloned java-app into /config/IdeaProjects.

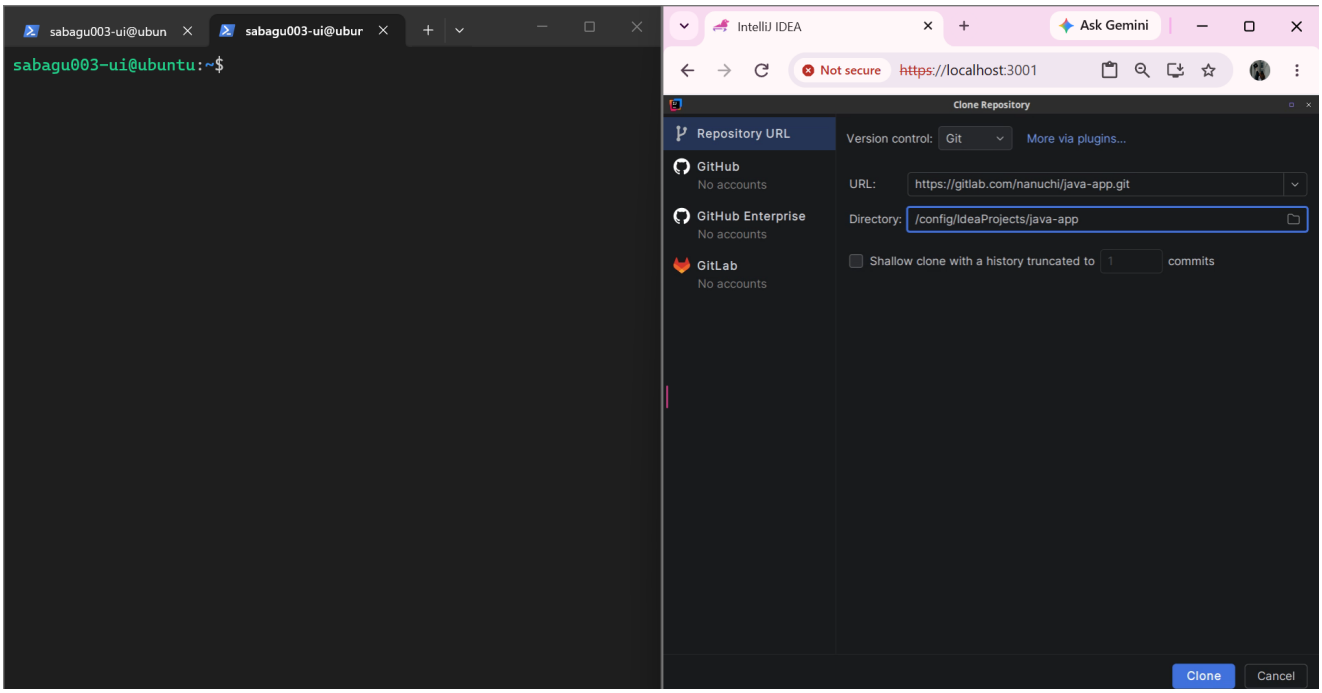


Figure 3.1: Clone dialog for java-app (gradle_project_clone.png)

Step 2: Built the project with the Gradle Wrapper (JDK 16).

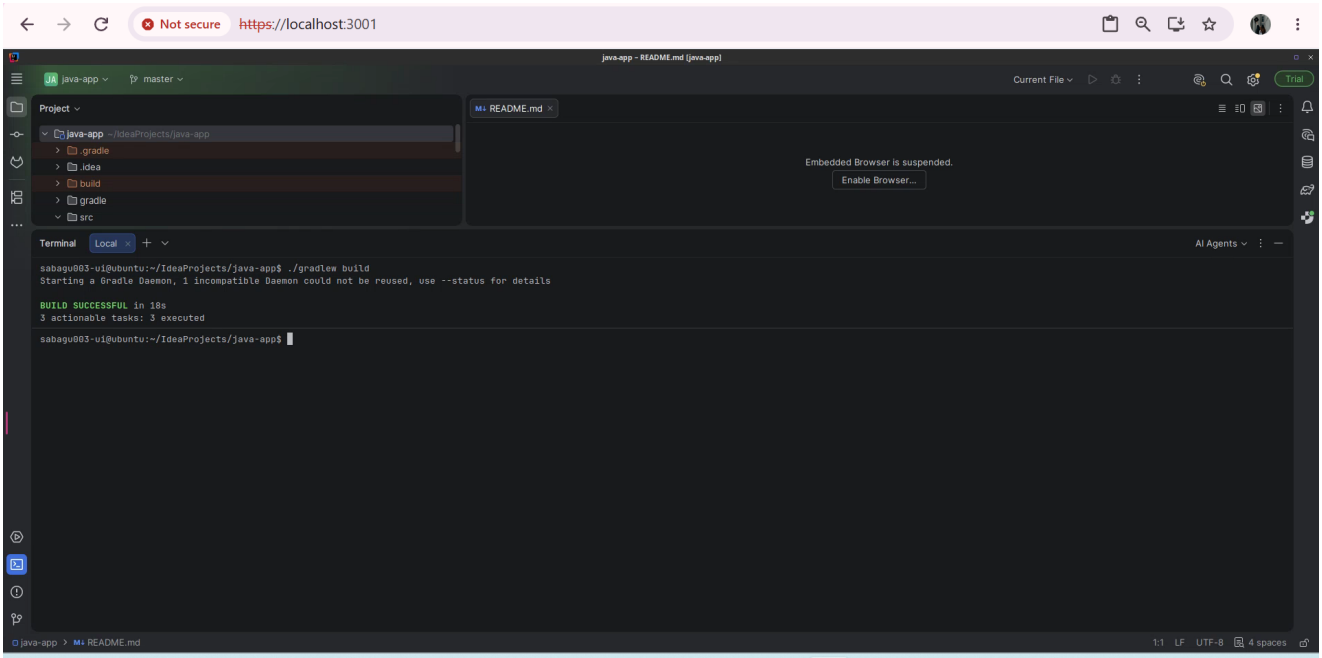


Figure 3.2: ./gradlew build - BUILD SUCCESSFUL (gradle_build.png)

Step 3: Added the Elasticsearch 7.0.0 dependency to build.gradle and synced Gradle.

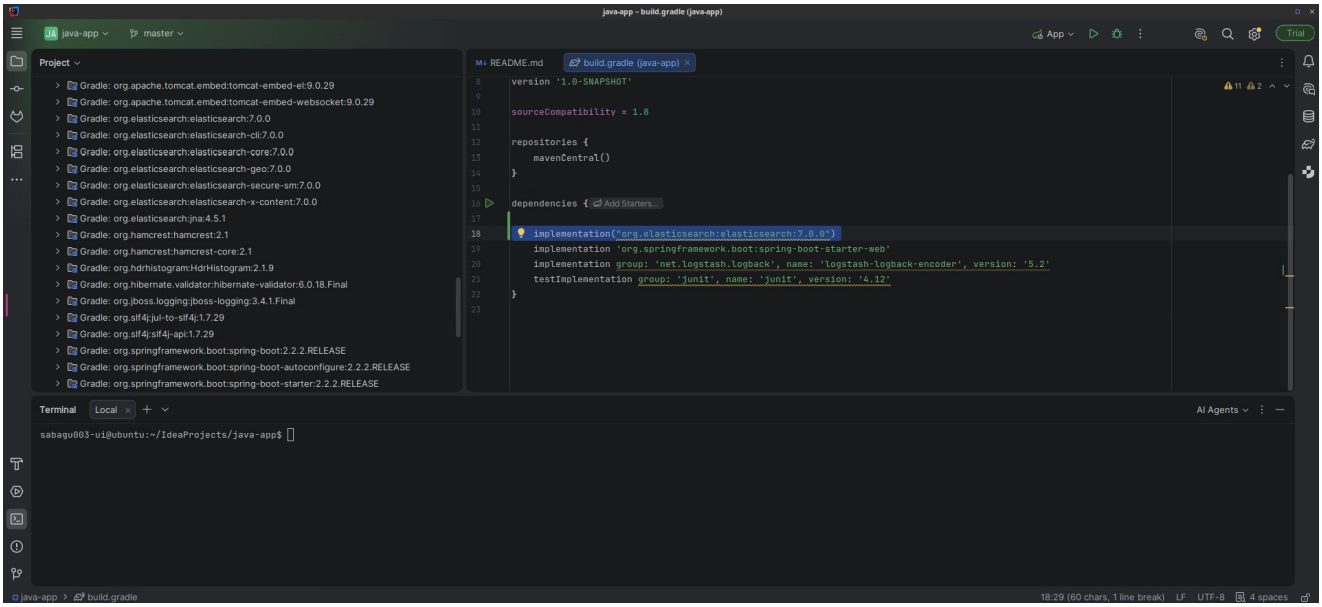


Figure 3.3: Elasticsearch 7.0.0 in build.gradle and External Libraries (gradle_elasticsearch_dependency.png)

Step 4: Rebuilt the project after adding the dependency.

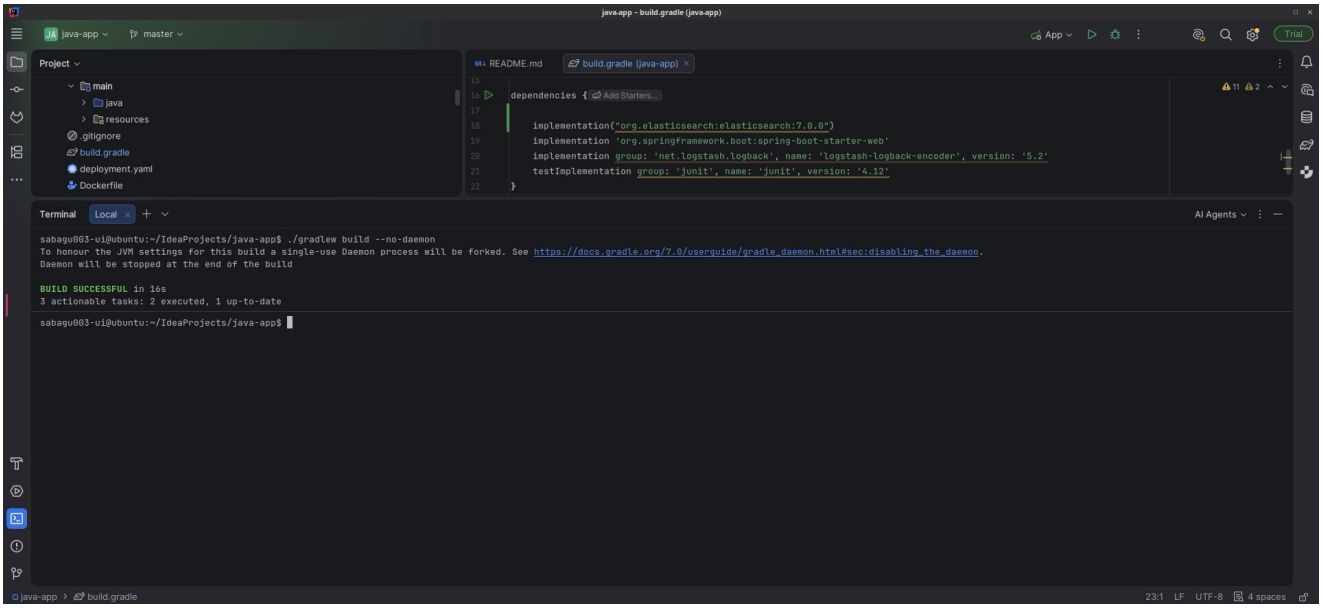


Figure 3.4: Gradle rebuild - BUILD SUCCESSFUL (gradle_rebuild.png)

Step 5: Found the generated JAR in build/libs.

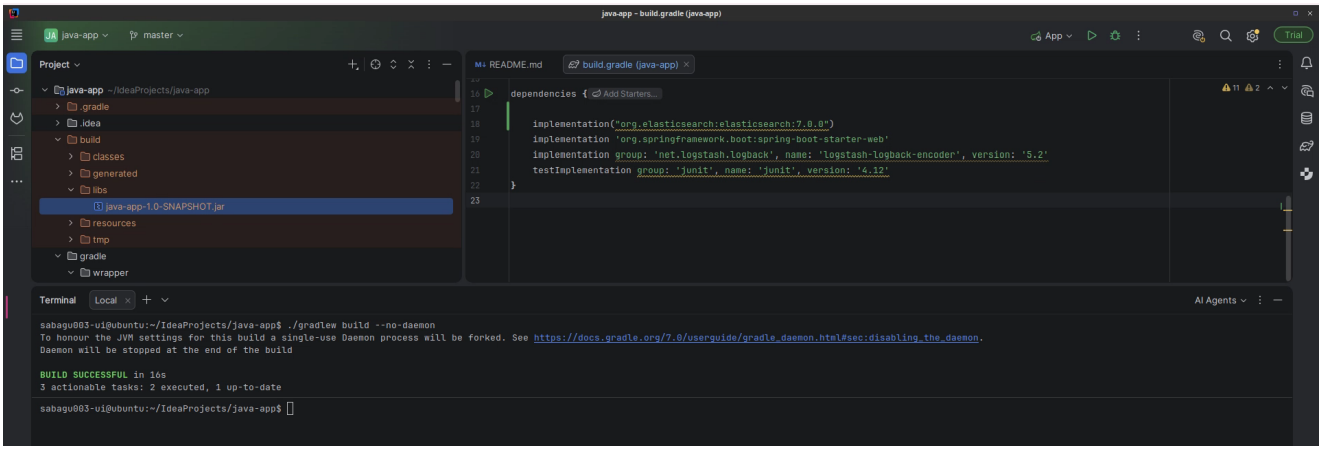


Figure 3.5: java-app-1.0-SNAPSHOT.jar in build/libs (gradle_jar.png)

Step 6: Ran the JAR with java -jar; the application started on port 8080.

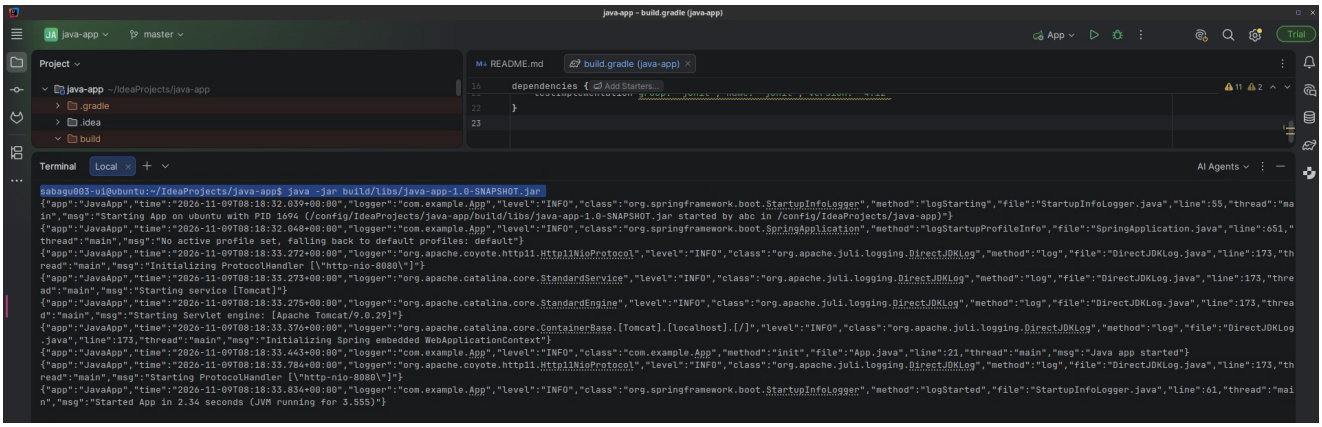


Figure 3.6: Gradle application running (gradle_running.png)

Step 7: Stopped the application with Ctrl+C.

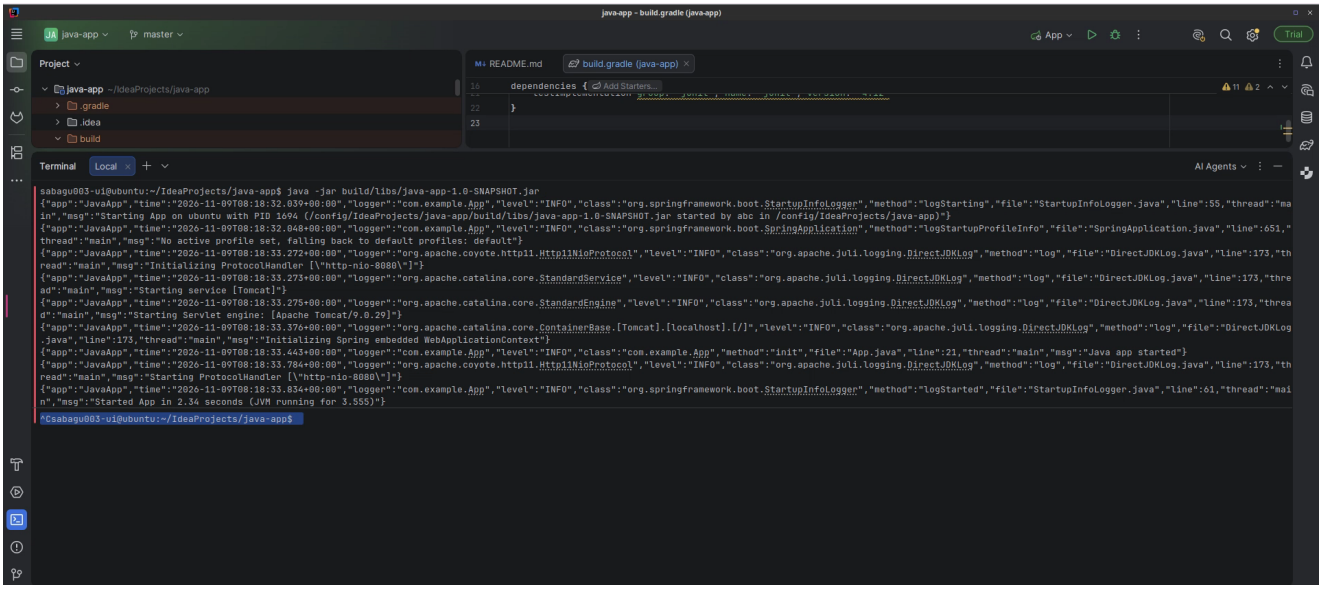


Figure 3.7: Gradle application stopped (gradle_stopped.png)

Step 8: Closed the project and returned to the Welcome screen.

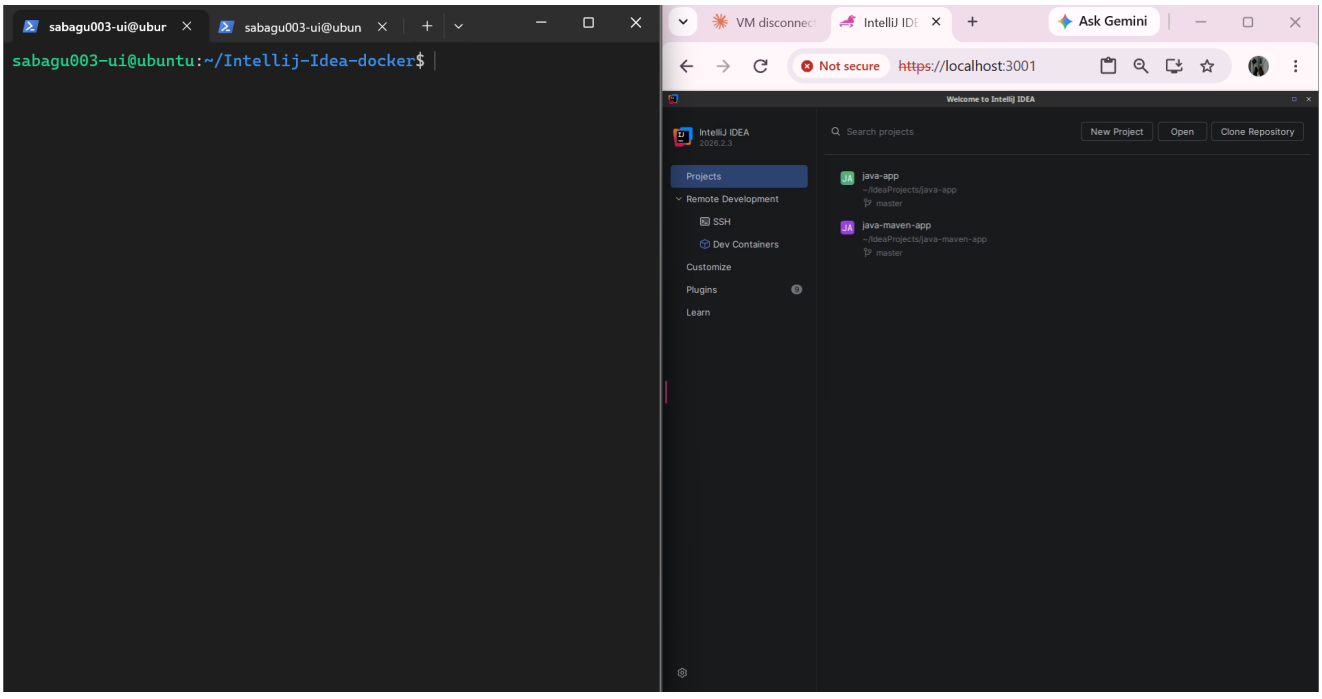


Figure 3.8: Gradle project closed (gradle_project_closed.png)

Task 4: Install and Run the Node.js Project

Step 1: Cloned node-project into /config/IdeaProjects.

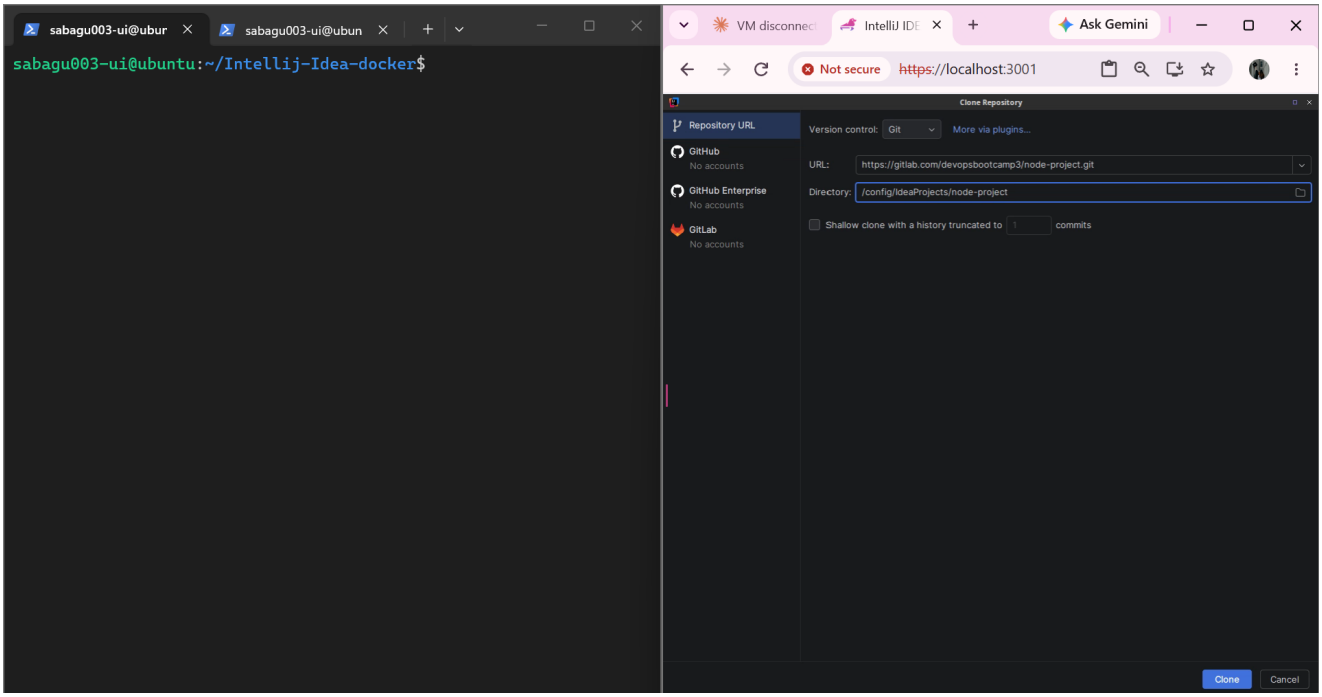


Figure 4.1: Clone dialog for node-project (node_project_clone.png)

Step 2: Installed Node.js and npm and checked both versions.

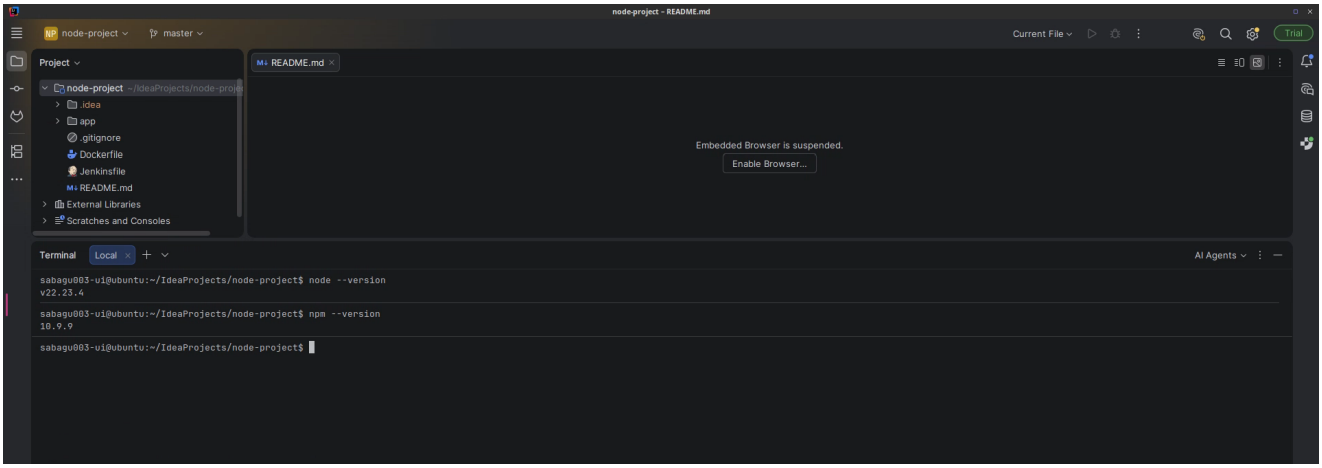


Figure 4.2: Node.js and npm installed (nodejs_installed.png)

Step 3: Installed the dependencies with npm install in the app directory.

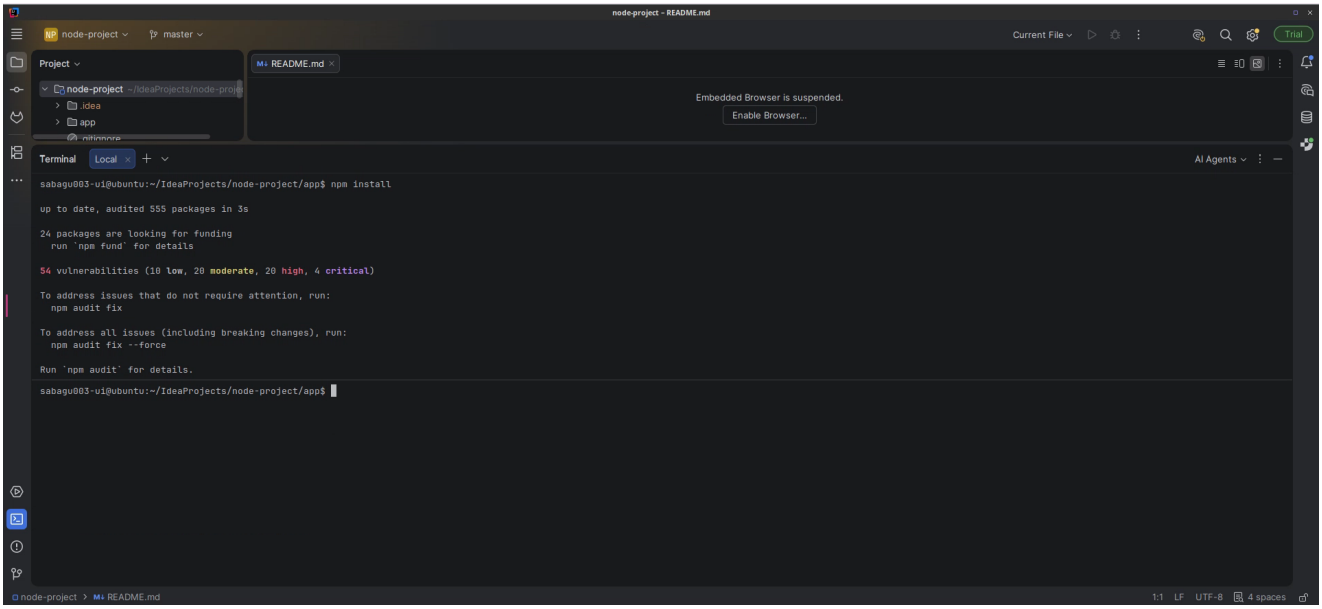


Figure 4.3: npm install completed (npm_install.png)

Step 4: Port 3000 was busy (EADDRINUSE), so I changed server.js to listen on port 4000 and ran npm start.

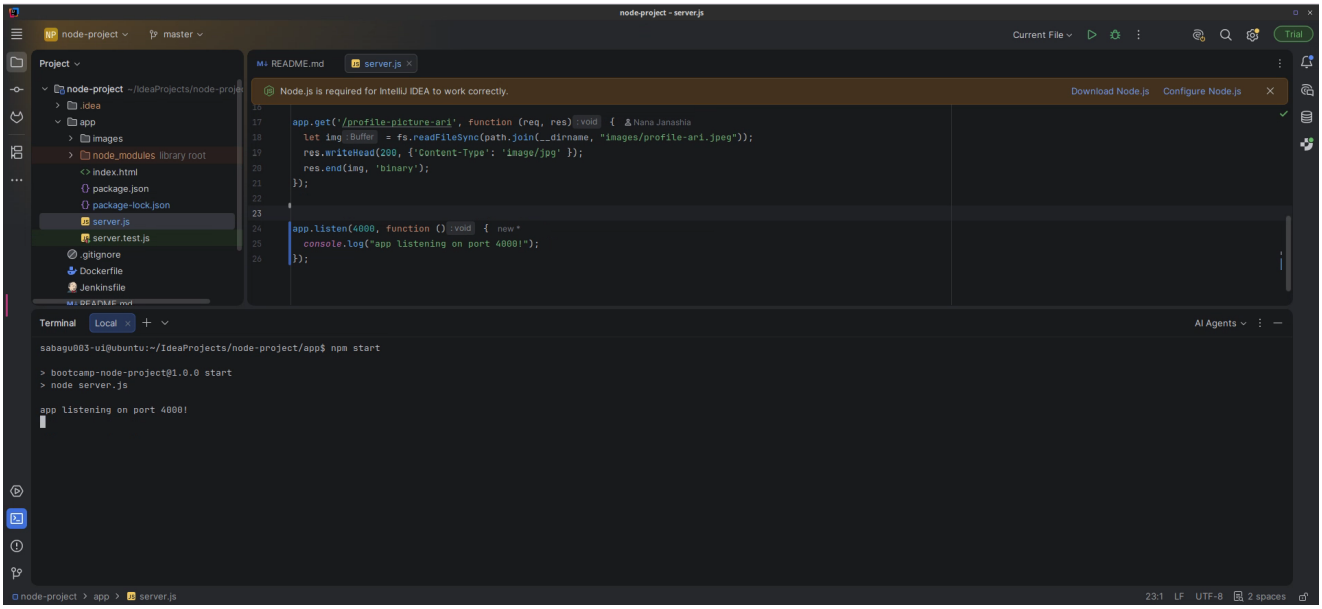


Figure 4.4: Application listening on port 4000 (node_running.png)

Step 5: Opened http://localhost:4000 in the container browser; the page and images load.

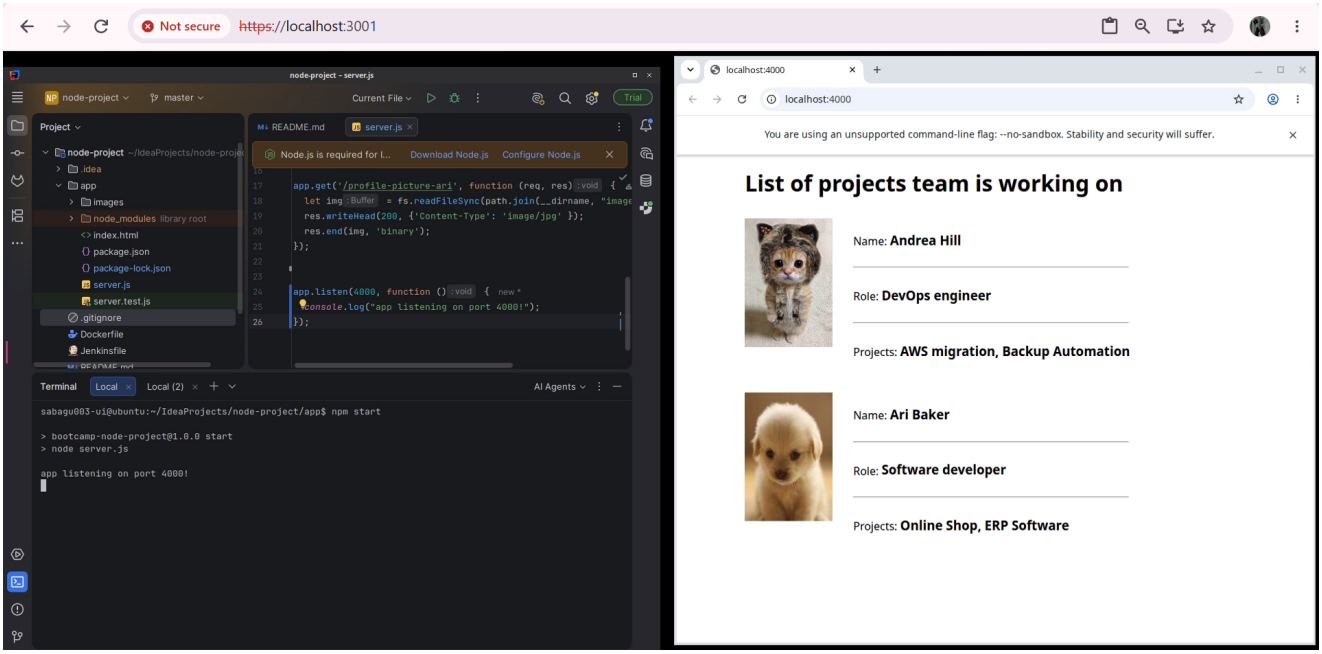


Figure 4.5: Node.js page in the container browser (node_browser.png)

Step 6: Ran the Jest test with npm test; it passed.

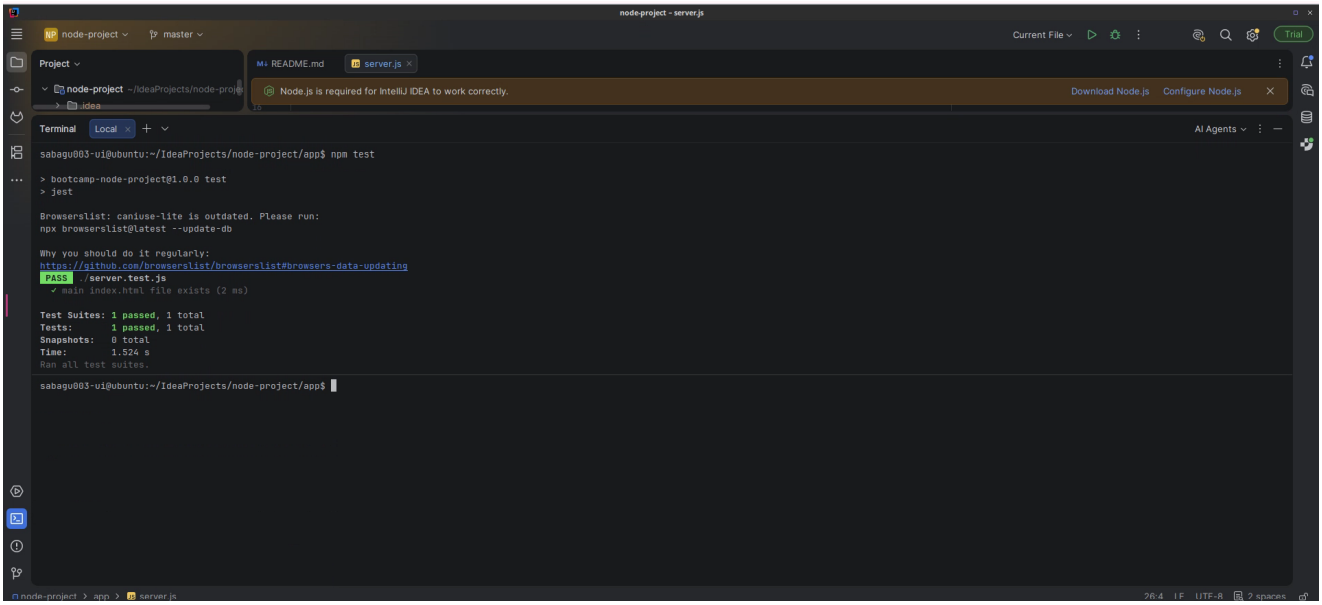


Figure 4.6: npm test - test passed (npm_test.png)

Step 7: Created the package tarball with npm pack.

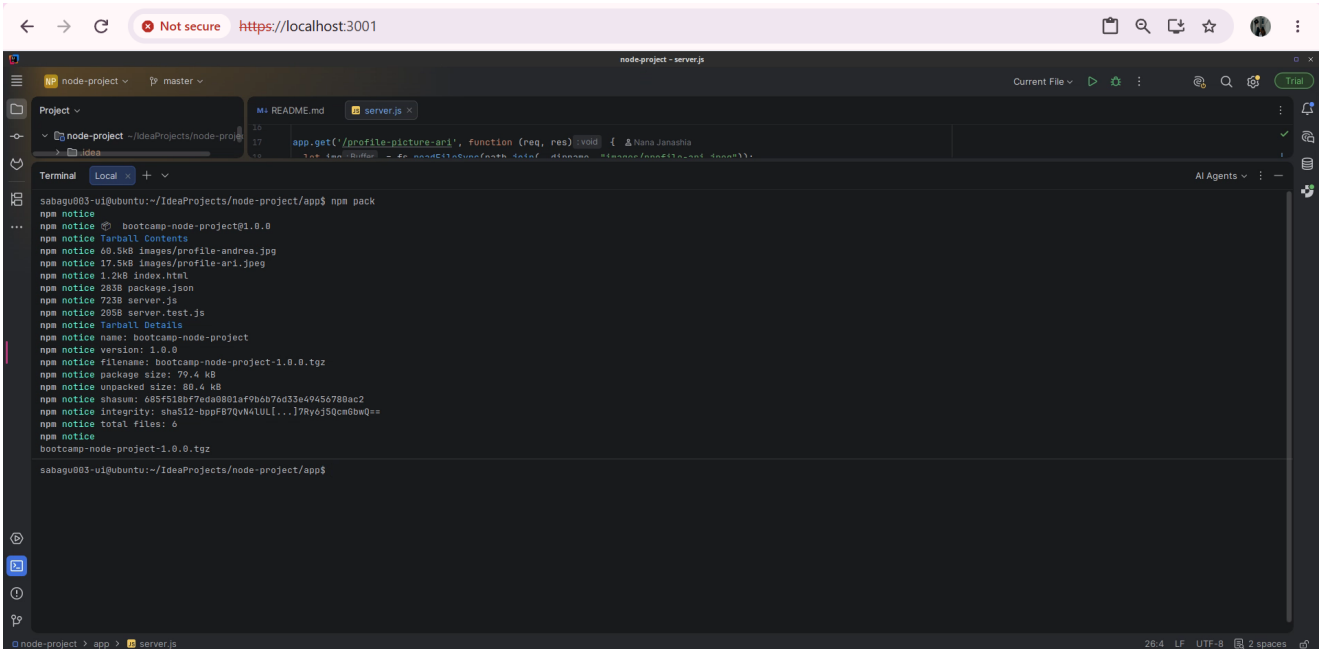


Figure 4.7: bootcamp-node-project-1.0.0.tgz created (npm_pack.png)

Task 5: Build the React and Node.js API Artifact

Step 1: Cloned react-nodejs-example into /config/IdeaProjects.

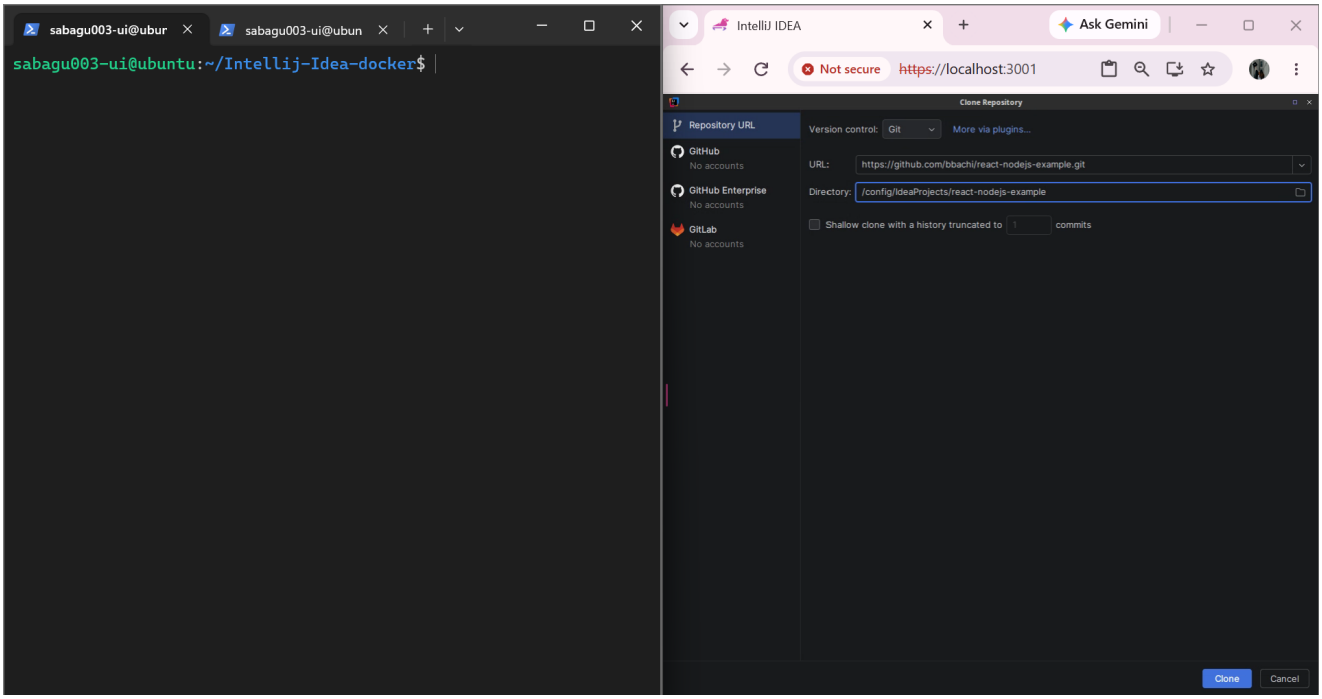


Figure 5.1: Clone dialog for react-nodejs-example (react_node_project_clone.png)

Step 2: Installed the API dependencies with npm install in the api directory.

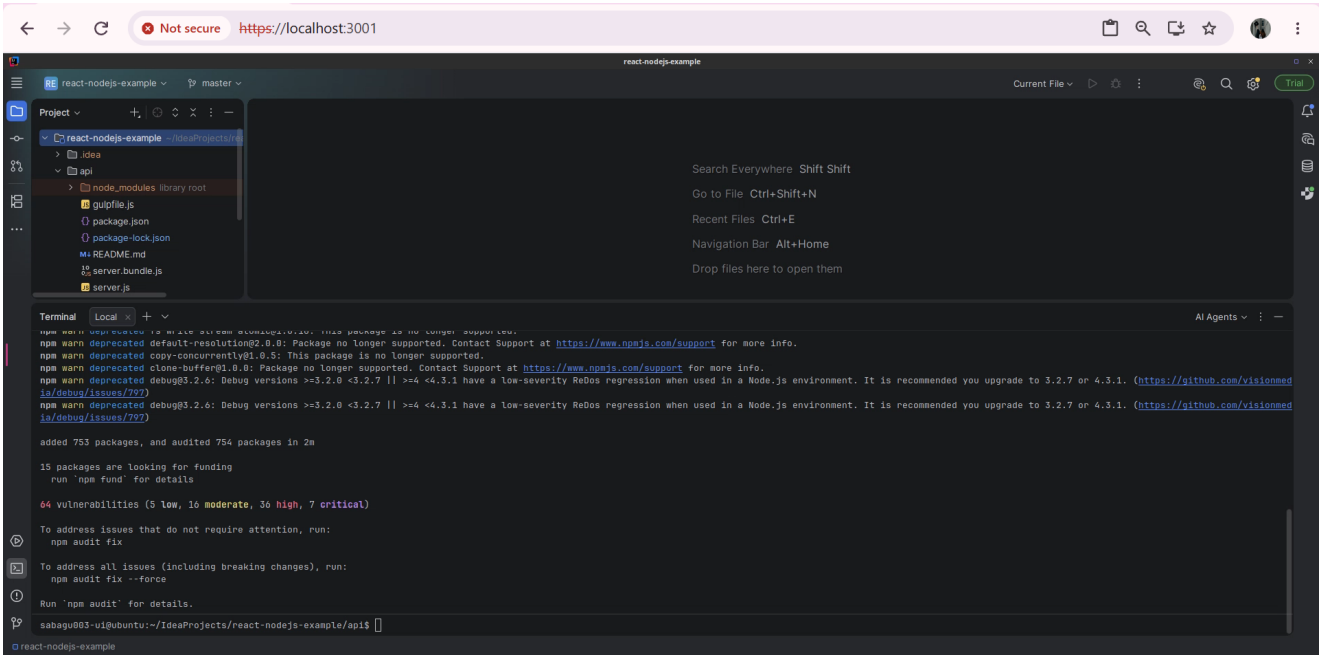


Figure 5.2: API dependencies installed (node_modules) (react_api_dependencies.png)

Step 3: Built the API with webpack using npm run build; server.bundle.js was created.

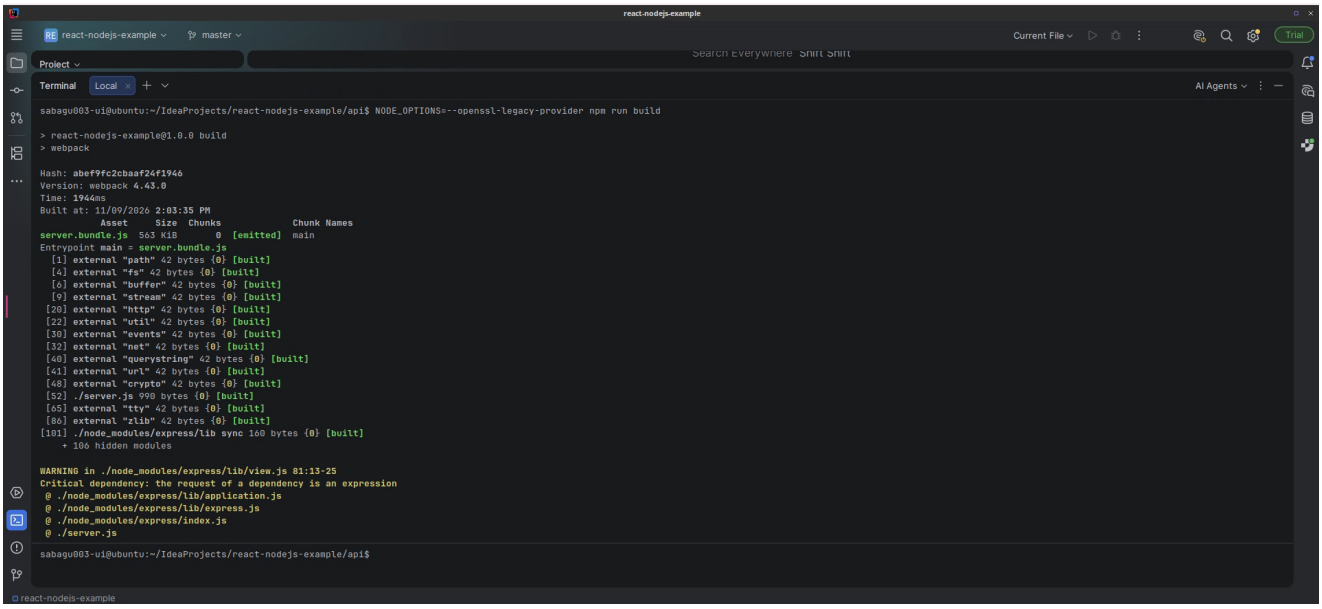


Figure 5.3: Webpack build output (server.bundle.js) (react_api_build.png)

Step 4: Viewed the bundle with cat server.bundle.js | more.

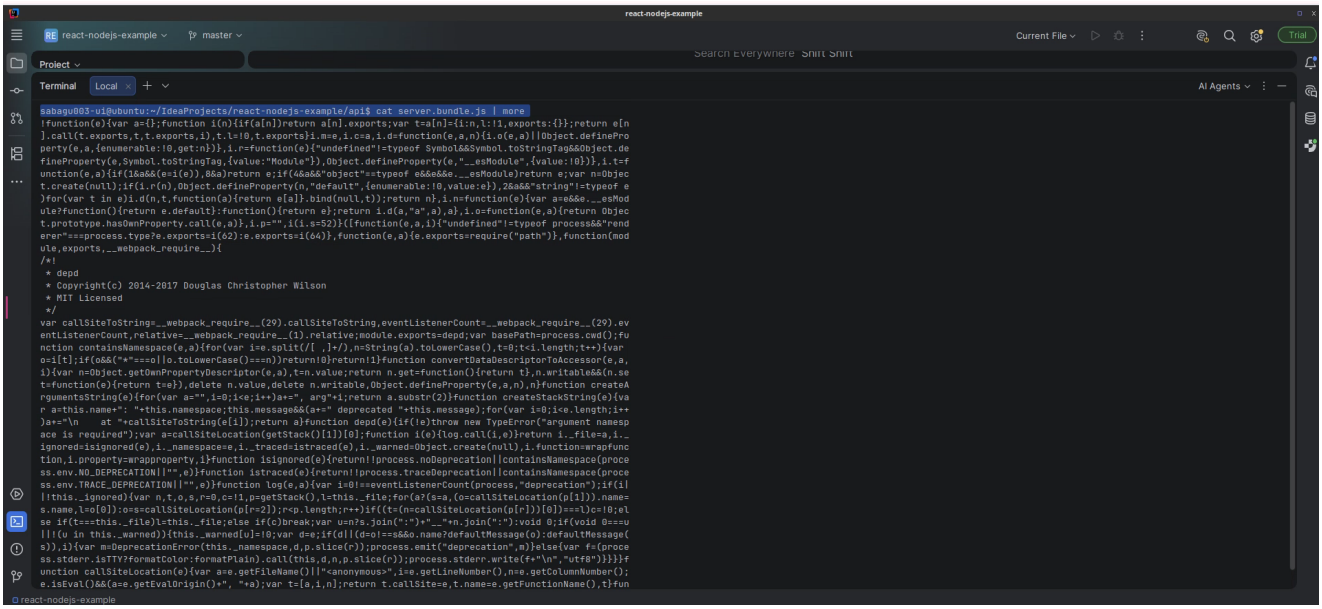


Figure 5.4: Bundled server.bundle.js (react_api_bundle.png)

Task 6: Build a Docker Image for the Node.js Project

Step 1: Checked that Docker is installed.

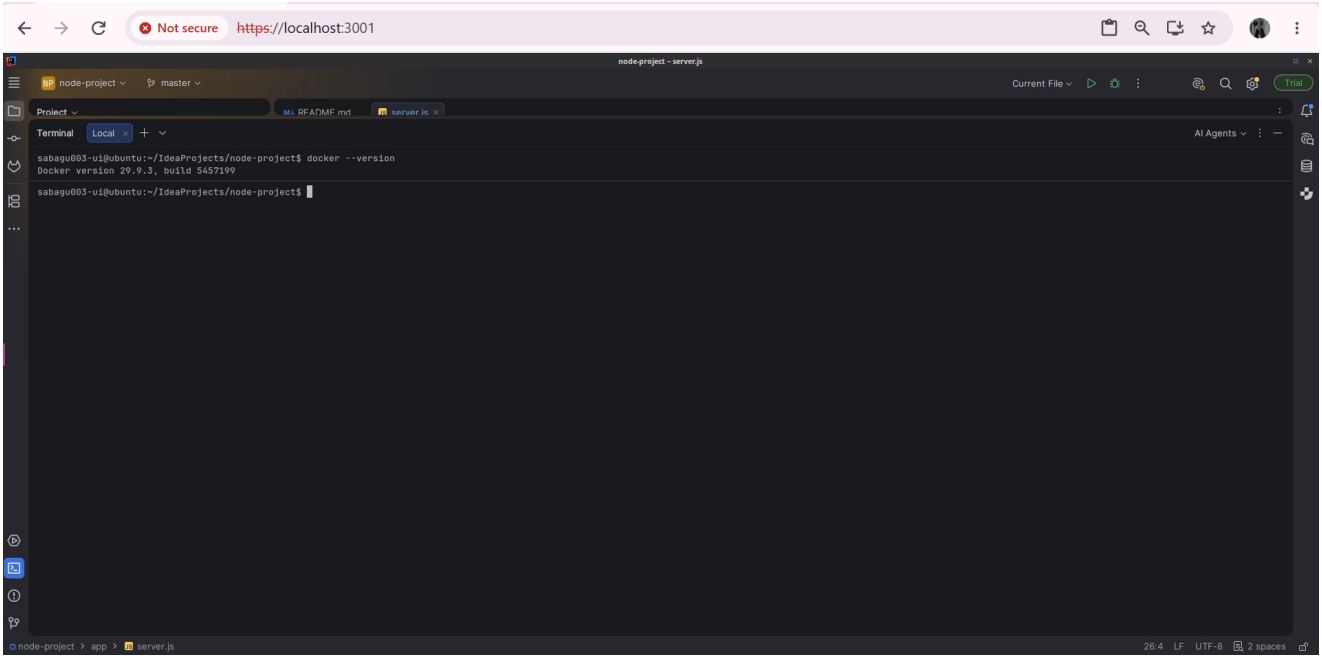


Figure 6.1: Docker version (docker_installed.png)

Step 2: Confirmed the Dockerfile exists in the project root.

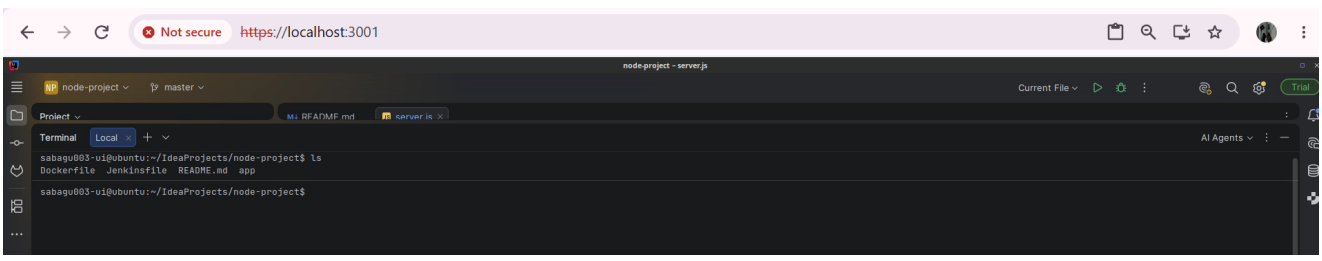


Figure 6.2: Dockerfile listed by ls (dockerfile_exists.png)

Step 3: Listed the Docker images before the build.

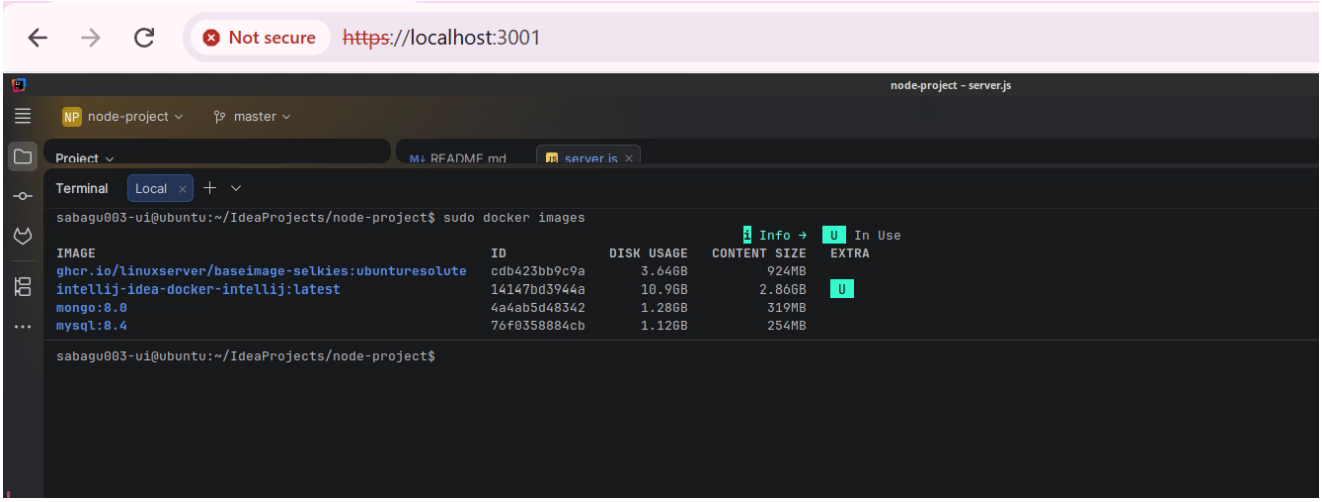


Figure 6.3: Images before the build (docker_images_before.png)

Step 4: Built the image with docker build -t lab5:1.0 .

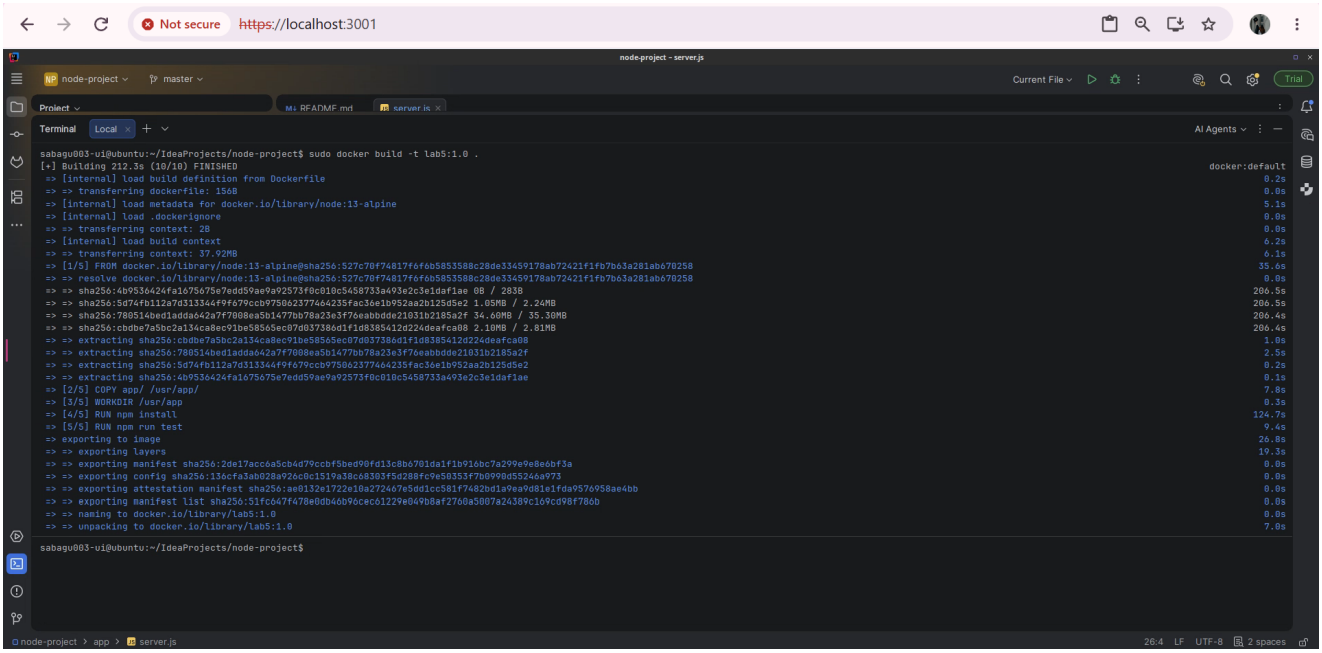


Figure 6.4: Docker image built and tagged lab5:1.0 (docker_build.png)

Step 5: Listed the images again; lab5 with tag 1.0 now appears.

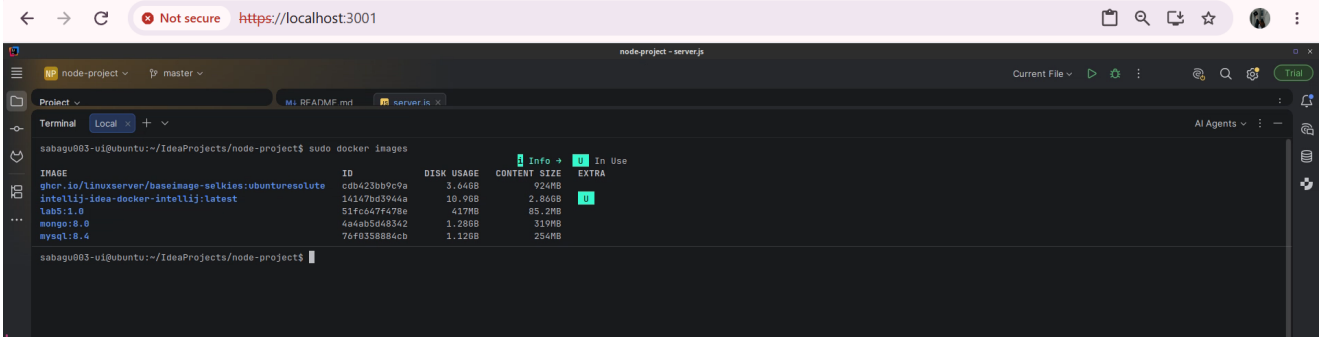


Figure 6.5: Images after the build (docker_images_after.png)

Exam Evaluation Questions (Practice)

1. Which values in pom.xml determine the JAR name, and what does the package phase do?

The artifactId (java-maven-app) and version (1.1.0-SNAPSHOT) give java-maven-app-1.1.0-SNAPSHOT.jar. The package phase runs the earlier phases (compile, test) and then packs the classes and resources into a JAR in target/; the Spring Boot repackage goal makes it executable.

2. Why can ./gradlew build work without a system-wide Gradle? Which JDK settings do you check for an unsupported Java version?

The Gradle Wrapper downloads the Gradle version the project needs on its first run. For an unsupported Java version (Gradle 7.x cannot run on Java 21), check Project SDK (JDK 16), the project language level, Settings > Gradle > Gradle JVM, and the Java shown by ./gradlew --version.

3. Why does EADDRINUSE occur on port 3000, and how does port 4000 let the container browser open the app?

The Webtop desktop in the container already uses port 3000, so the app cannot bind to it. Port 4000 is free, and the browser inside the same container reaches the app at http://localhost:4000.